If you like, next I can draft concrete rule pseudocode (or Python function signatures) that you can drop into your project.

Yes, let's define concrete, copy-paste-ready function signatures and pseudocode you can turn into Python.

1. Data model assumptions

Assume you've already loaded CSVs into Pandas and a NetworkX graph:

```
import pandas as pd
import networkx as nx

qmgrs_df    = pd.read_csv("qmgrs.csv")      # columns: qmgr_id, owner_team, ...
queues_df   = pd.read_csv("queues.csv")     # columns: queue_id, qmgr_id, type(local/rem
channels_df = pd.read_csv("channels.csv")   # columns: channel_id, from_qmgr, to_qmgr, t
apps_df     = pd.read_csv("apps.csv")       # columns: app_id, qmgr_id, role(producer/c
rels_df     = pd.read_csv("relations.csv")  # columns: app_id, queue_id, role(producer/

G = nx.DiGraph()
for _, row in channels_df.iterrows():
    G.add_edge(row["from_qmgr"], row["to_qmgr"], channel_id=row["channel_id"])
```

A simple structure for rule results:

```
from dataclasses import dataclass

@dataclass
class RuleChange:
    rule_id: str
    object_type: str
    object_id: str
    action: str      # "create", "update", "delete"
    reason: str
```

2. Complexity metric function

```
def compute_complexity(G, producers, consumers,
                       w1=1, w2=2, w3=1, w4=3):
    """
    producers, consumers: dict[app_id] -> qmgr_id
    Returns: dict with components and total score.
    """
    E = G.number_of_edges()

    # Average hop count between producer and consumer qmgrs
    hop_lengths = []
    for app_id in producers.keys():
        src = producers[app_id]
        dst = consumers.get(app_id)
        if src and dst and nx.has_path(G, src, dst):
            hop_lengths.append(nx.shortest_path_length(G, src, dst))
    H = sum(hop_lengths) / len(hop_lengths) if hop_lengths else 0

    # Max fan-out
    F = max(dict(G.out_degree()).values()) if G.number_of_nodes() else 0

    # Cycle count (simple approximation)
    cycles = list(nx.simple_cycles(G))
    C = len(cycles)

    score = w1 * E + w2 * H + w3 * F + w4 * C
    return {"E": E, "H": H, "F": F, "C": C, "score": score}
```

3. Rule 1 – One queue manager per app

```
def rule_one_qmgr_per_app(apps_df, preferred_qmgr_col="qmgr_id"):
    """
    Enforce exactly one qmgr per app.
    Returns: updated apps_df, list[RuleChange]
    """
    changes = []
    # Example: if duplicates exist, keep first as preferred
    for app_id, group in apps_df.groupby("app_id"):
        if len(group["qmgr_id"].unique()) > 1:
            preferred_qmgr = group.iloc[0]["qmgr_id"]
            mask = (apps_df["app_id"] == app_id) & (apps_df["qmgr_id"] != preferred_qmg
            apps_df.loc[mask, "qmgr_id"] = preferred_qmgr
            changes.append(
                RuleChange(
                    rule_id="R1_ONE_QM_PER_APP",
                    object_type="app",
                    object_id=str(app_id),
                    action="update",
                    reason=f"App connected to multiple QMgrs; normalized to {preferred_c
                )
```

```
            )
    return apps_df, changes
```

4. Rule 2 – Producers write to local queues

```
def rule_producer_local_queue(apps_df, rels_df, queues_df):
    """
    Ensure producers write to local queues on their own qmgr.
    For violations, create local + remote + xmit queues (conceptually).
    Returns: updated queues_df, rels_df, list[RuleChange]
    """
    changes = []
    # Build helper maps
    app_qmgr = dict(zip(apps_df["app_id"], apps_df["qmgr_id"]))
    queue_qmgr = dict(zip(queues_df["queue_id"], queues_df["qmgr_id"]))

    for _, rel in rels_df[rels_df["role"] == "producer"].iterrows():
        app_id = rel["app_id"]
        q_id   = rel["queue_id"]
        app_qm = app_qmgr.get(app_id)
        q_qm   = queue_qmgr.get(q_id)

        if app_qm is None or q_qm is None:
            continue

        if app_qm != q_qm:
            # violation: producer writing to non-local queue
            # create new local queue id
            new_local_id  = f"{q_id}_LOCAL_{app_qm}"
            new_remote_id = f"{q_id}_REMOTE_{app_qm}"
            new_xmit_id   = f"{q_id}_XMIT_{app_qm}"

            # Append to queues_df (pseudo, adjust for real code)
            for qid, qtype, remote_qmgr in [
                (new_local_id,  "local",  None),
                (new_remote_id, "remote", q_qm),
                (new_xmit_id,   "xmit",   q_qm),
            ]:
                queues_df.loc[len(queues_df)] = {
                    "queue_id": qid,
                    "qmgr_id": app_qm,
                    "type": qtype,
                    "remote_qmgr_id": remote_qmgr
                }

            # Repoint producer relation to new local queue
            rels_df.loc[(rels_df["app_id"] == app_id) &
                        (rels_df["queue_id"] == q_id) &
                        (rels_df["role"] == "producer"), "queue_id"] = new_local_id

            changes.append(
                RuleChange(
                    rule_id="R2_PRODUCER_LOCAL",
                    object_type="relation",
```

```
                object_id=f"{app_id}->{q_id}",
                action="update",
                reason=f"Producer {app_id} now writes to local queue {new_local_id} o
                       f"remote and XMIT queues created for QMgr {q_qm}."
            )
        )

    return queues_df, rels_df, changes
```

5. Rule 3 – Consumers read from local queues

```
def rule_consumer_local_queue(apps_df, rels_df, queues_df):
    """
    Ensure consumers read from local queues on their own qmgr.
    Returns: updated queues_df, rels_df, list[RuleChange]
    """
    changes = []
    app_qmgr = dict(zip(apps_df["app_id"], apps_df["qmgr_id"]))
    queue_qmgr = dict(zip(queues_df["queue_id"], queues_df["qmgr_id"]))

    for _, rel in rels_df[rels_df["role"] == "consumer"].iterrows():
        app_id = rel["app_id"]
        q_id   = rel["queue_id"]
        app_qm = app_qmgr.get(app_id)
        q_qm   = queue_qmgr.get(q_id)

        if app_qm is None or q_qm is None:
            continue

        if app_qm != q_qm:
            new_local_id = f"{q_id}_CONSUMER_LOCAL_{app_qm}"
            # Create local queue
            queues_df.loc[len(queues_df)] = {
                "queue_id": new_local_id,
                "qmgr_id": app_qm,
                "type": "local",
                "remote_qmgr_id": None
            }
            # Rewire consumer relation
            rels_df.loc[(rels_df["app_id"] == app_id) &
                        (rels_df["queue_id"] == q_id) &
                        (rels_df["role"] == "consumer"), "queue_id"] = new_local_id

            changes.append(
                RuleChange(
                    rule_id="R3_CONSUMER_LOCAL",
                    object_type="relation",
                    object_id=f"{app_id}->{q_id}",
                    action="update",
                    reason=f"Consumer {app_id} now reads from local queue {new_local_id}
                )
            )
```

```
    return queues_df, rels_df, changes
```

6. Rule 4 – Channel consolidation

```
def rule_consolidate_channels(channels_df):
    """
    If multiple sender channels exist between same QMgr pair, keep one.
    Returns: updated channels_df, list[RuleChange]
    """
    changes = []
    key_cols = ["from_qmgr", "to_qmgr", "type"]
    groups = channels_df.groupby(key_cols)

    to_drop = []
    for key, group in groups:
        if len(group) > 1 and key[2] == "sender":
            # keep first, drop rest
            keep_id = group.iloc[0]["channel_id"]
            for _, row in group.iloc[1:].iterrows():
                to_drop.append(row["channel_id"])
                changes.append(
                    RuleChange(
                        rule_id="R4_CONSOLIDATE_CHANNELS",
                        object_type="channel",
                        object_id=str(row["channel_id"]),
                        action="delete",
                        reason=f"Duplicate sender channel between {key[0]} and {key[1]} co
                    )
                )

    if to_drop:
        channels_df = channels_df[~channels_df["channel_id"].isin(to_drop)].reset_index(d
    return channels_df, changes
```

7. Rule 5 – Remove unused objects

```
def rule_cleanup_unused(queues_df, channels_df, rels_df):
    """
    Remove queues/channels with no producer or consumer usage.
    Returns: updated dataframes, list[RuleChange]
    """
    changes = []

    used_queues = set(rels_df["queue_id"].unique())
    unused_queues = set(queues_df["queue_id"]) - used_queues
    if unused_queues:
        queues_df = queues_df[~queues_df["queue_id"].isin(unused_queues)].reset_index(dr
        for qid in unused_queues:
            changes.append(
                RuleChange(
```

```
                    rule_id="R5_CLEANUP_UNUSED",
                    object_type="queue",
                    object_id=str(qid),
                    action="delete",
                    reason="Queue has no producers or consumers after refactor."
                )
            )

    used_qmgrs = set(queues_df["qmgr_id"]) | set(channels_df["from_qmgr"]) | set(channel
    # Optional: drop QMgrs not in used_qmgrs, if you have a qmgrs_df

    return queues_df, channels_df, changes
```

8. Orchestrator

```
def apply_all_rules(apps_df, rels_df, queues_df, channels_df):
    all_changes = []

    apps_df, ch = rule_one_qmgr_per_app(apps_df)
    all_changes.extend(ch)

    queues_df, rels_df, ch = rule_producer_local_queue(apps_df, rels_df, queues_df)
    all_changes.extend(ch)

    queues_df, rels_df, ch = rule_consumer_local_queue(apps_df, rels_df, queues_df)
    all_changes.extend(ch)

    channels_df, ch = rule_consolidate_channels(channels_df)
    all_changes.extend(ch)

    queues_df, channels_df, ch = rule_cleanup_unused(queues_df, channels_df, rels_df)
    all_changes.extend(ch)

    return apps_df, rels_df, queues_df, channels_df, all_changes
```

If you share (or describe) the actual CSV column names, I can adapt this pseudocode to match them exactly so you can paste and tweak with minimal effort.